

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐẠI NAM



AUTHENTICATED IOT CLOUD INGESTION

Nguyễn Như Thành — MSV: 1871020536

Nguyễn Thị Tuyết — MSV: 1871020237

Đinh Hùng Thiện — MSV: 1871020638

BÁO CÁO BÀI TẬP LỚN

HÀ NỘI, NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐẠI NAM



Nguyễn Như Thành — MSV: 1871020536

Nguyễn Thị Tuyết — MSV: 1871020237

Đinh Hùng Thiện — MSV: 1871020638

AUTHENTICATED IOT CLOUD INGESTION

BÁO CÁO BÀI TẬP LỚN

GIẢNG VIÊN HƯỚNG DẪN: ThS. Lê Thị Thùy Trang

HÀ NỘI, NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan báo cáo bài tập lớn học phần *Nhập môn An toàn Bảo mật Thông tin* (Đề tài 10: Authenticated IoT Cloud Ingestion) này là kết quả tìm hiểu, thiết kế và triển khai của cá nhân tôi.

Toàn bộ mã nguồn (firmware ESP32, máy chủ Cloud, công cụ kiểm thử), số liệu, hình ảnh và nội dung trình bày trong báo cáo là trung thực, có nguồn gốc rõ ràng, và các tài liệu tham khảo được trích dẫn đầy đủ theo đúng quy định. Tôi xin chịu trách nhiệm về tính chính xác và trung thực của toàn bộ nội dung báo cáo.

Hà Nội, ngày 5 tháng 7 năm 2026

Tác giả

NGUYỄN NHƯ THÀNH

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành tới Ban Giám hiệu Trường Đại học Đại Nam, Khoa Công nghệ thông tin, cùng quý thầy cô đã giảng dạy và tạo điều kiện cho em được thực hiện bài tập lớn học phần *Nhập môn An toàn Bảo mật Thông tin*.

Em xin cảm ơn các giảng viên hướng dẫn ThS. Lê Thị Thùy Trang đã tận tình hướng dẫn, góp ý về hướng tiếp cận giao thức bảo mật, chỉ ra các yêu cầu kiểm thử bắt buộc, và hỗ trợ em trong suốt quá trình thực hiện đề tài.

Mặc dù đã cố gắng tìm hiểu tài liệu tham khảo (tiêu chuẩn mật mã học, tài liệu kỹ thuật của các thư viện/phần cứng sử dụng) và thử nghiệm hệ thống một cách cẩn thận, song do kiến thức và kinh nghiệm thực tế còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Em kính mong nhận được ý kiến góp ý của quý thầy cô để hoàn thiện hệ thống và báo cáo hơn nữa.

Hà Nội, ngày ...tháng ...năm 2026

Tác giả

NGUYỄN NHƯ THÀNH

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

TỪ VIẾT TẮT	TÊN ĐẦY ĐỦ CỦA TỪ VIẾT TẮT
IoT	Internet of Things — Mạng lưới vạn vật kết nối
AES	Advanced Encryption Standard — Chuẩn mã hoá tiên tiến
GCM	Galois/Counter Mode — Chế độ mã hoá xác thực của AES
AEAD	Authenticated Encryption with Associated Data — Mã hoá kết hợp xác thực
RSA	Rivest–Shamir–Adleman — Thuật toán mã hoá khoá công khai
OAEP	Optimal Asymmetric Encryption Padding — Đệm mã hoá bất đối xứng tối ưu
PKCS#1	Public-Key Cryptography Standards #1 — Chuẩn mật mã khoá công khai số 1
SHA	Secure Hash Algorithm — Thuật toán băm bảo mật
MITM	Man-in-the-Middle — Tấn công đứng giữa
TCP	Transmission Control Protocol — Giao thức điều khiển truyền vận
JSON	JavaScript Object Notation — Định dạng trao đổi dữ liệu dạng văn bản
ACK / NACK	Acknowledgement / Negative Acknowledgement — Phản hồi chấp nhận/từ chối
NFC	Near-Field Communication — Giao tiếp trường gần
RFID	Radio-Frequency Identification — Nhận dạng qua tần số vô tuyến
NTP	Network Time Protocol — Giao thức đồng bộ thời gian mạng
API	Application Programming Interface — Giao diện lập trình ứng dụng
CSDL	Cơ sở dữ liệu

DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH

BẢNG:

Tên bảng	Trang
Bảng 1.1. Đối chiếu yêu cầu bảo mật và giải pháp áp dụng	2
Bảng 3.1. Thời gian xử lý mật mã theo kích thước gói tin (điền số liệu thực đo).....	15
Bảng 3.2. Đặc điểm các thuật toán sử dụng	15
Bảng .3. Đối chiếu yêu cầu báo cáo với cấu trúc tài liệu.....	18

SƠ ĐỒ:

Tên sơ đồ	Trang
Sơ đồ 1.1. Kiến trúc Gateway-Node của hệ thống	3

HÌNH:

Tên hình	Trang

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT	iii
DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH	iv
LỜI MỞ ĐẦU	vi
CHƯƠNG 1. ĐẶT VẤN ĐỀ VÀ KIẾN TRÚC HỆ THỐNG	1
1.1. Đặt vấn đề và phân tích yêu cầu bài toán	1
1.2. Kiến trúc hệ thống	2
1.2.1. Kiến trúc phân cứng và hạ tầng mạng	2
1.2.2. Tham số vận hành và mô hình xử lý đồng thời	4
1.2.3. Thiết kế cấu trúc gói tin JSON	5
CHƯƠNG 2. THIẾT KẾ THUẬT TOÁN VÀ PHÂN TÍCH MÃ NGUỒN	7
2.1. Mô tả thuật toán — ba bước bảo mật	7
2.1.1. Bước 1 — Bắt tay (Handshake)	7
2.1.2. Bước 2 — Xác thực (Ký số và trao đổi khoá)	7
2.1.3. Bước 3 — Truyền dữ liệu và kiểm tra toàn vẹn	8
2.2. Phân tích mã nguồn	8
2.2.1. Firmware ESP32 (thư mục <i>iot/</i>)	8
2.2.2. Cloud Server (thư mục <i>cloud_server/</i>)	9
2.2.3. Công cụ hỗ trợ	11
CHƯƠNG 3. THỬ NGHIỆM, ĐÁNH GIÁ VÀ ĐỀ XUẤT CẢI TIẾN	12
3.1. Thử nghiệm — ba kịch bản đối chứng	12
3.1.1. Luồng 1 — Hệ thống hoạt động hợp pháp (Happy Path)	12
3.1.2. Luồng 2 — Hệ thống không bảo mật (trường hợp đối chứng — Baseline)	12
3.1.3. Luồng 3 — Kịch bản tấn công can thiệp nâng cao (bẻ gãy đòn tấn công)	13
3.2. Đánh giá hiệu quả	14
3.3. Phân tích, nhận xét đặc điểm các thuật toán sử dụng	15
3.4. Đề xuất cải tiến và hướng phát triển	16
KẾT LUẬN	17
PHỤ LỤC	18
DANH MỤC TÀI LIỆU THAM KHẢO	20

LỜI MỞ ĐẦU

Trong bối cảnh các hệ thống Internet of Things (IoT) ngày càng được ứng dụng rộng rãi trong kiểm soát ra vào, giám sát an ninh và tự động hoá toà nhà, việc truyền tải dữ liệu cảm biến từ thiết bị biên (edge device) lên nền tảng đám mây qua mạng công cộng đặt ra yêu cầu cấp thiết về an toàn thông tin. Nếu dữ liệu được truyền ở dạng thô, kẻ tấn công có thể nghe lén, giả mạo hoặc chỉnh sửa nội dung để che giấu hành vi xâm nhập mà hệ thống giám sát không hề hay biết.

Báo cáo này trình bày kết quả thực hiện bài tập lớn học phần *Nhập môn An toàn Bảo mật Thông tin* — Đề tài 10: Authenticated IoT Cloud Ingestion: xây dựng và thử nghiệm một hệ thống Gateway-Node giả lập việc thu thập dữ liệu kiểm soát ra vào (cảm biến siêu âm HC-SR04 và mô-đun NFC PN532) từ thiết bị ESP32, mã hoá và xác thực dữ liệu bằng các thuật toán mật mã học (AES-GCM, RSA, SHA-512), truyền qua Socket TCP tới máy chủ Cloud (giả lập trên Raspberry Pi 5), đồng thời đánh giá hiệu quả an toàn thông tin của hệ thống thông qua các kịch bản đối chứng thực nghiệm (tấn công giả mạo dữ liệu, tấn công can thiệp đường truyền).

Mục tiêu của báo cáo là trình bày có hệ thống toàn bộ quá trình: từ phân tích yêu cầu bài toán, thiết kế giao thức bảo mật ba bước (bắt tay, xác thực và trao khoá, mã hoá và kiểm tra toàn vẹn), triển khai mã nguồn trên cả thiết bị IoT (ESP32, C++/mbedTLS) và máy chủ Cloud (Python), cho tới việc kiểm thử, đánh giá hiệu quả và đề xuất hướng phát triển. Qua đó, báo cáo cũng làm rõ những giới hạn thực tế của các thuật toán được chỉ định trong đề bài (đặc biệt là ràng buộc kích thước khoá RSA-1024 khi kết hợp với OAEP) và cách xử lý các giới hạn đó một cách có cơ sở khoa học.

Chương 1

ĐẶT VẤN ĐỀ VÀ KIẾN TRÚC HỆ THỐNG

1.1 Đặt vấn đề và phân tích yêu cầu bài toán

Các hệ thống IoT kiểm soát ra vào (cửa quét thẻ, cảm biến chuyển động, camera giám sát...) ngày càng được triển khai rộng rãi tại các toà nhà văn phòng, ký túc xá và cơ sở hạ tầng trọng yếu. Tuy nhiên, nếu dữ liệu cảm biến được truyền ở dạng thô (plaintext) qua mạng công cộng, hệ thống sẽ tồn tại lỗ hổng nghiêm trọng: kẻ tấn công có thể nghe lén, giả mạo hoặc chen dữ liệu trạng thái để che giấu hành vi đột nhập mà hệ thống giám sát không hề hay biết. Đây là dạng tấn công điển hình trong danh mục lỗ hổng IoT được OWASP ghi nhận, bao gồm thiếu cơ chế xác thực thiết bị và thiếu bảo vệ dữ liệu khi truyền tải [6]. Lỗ hổng này được minh chứng trực quan trong kịch bản đối chứng ở Luồng 2, trình bày ở Chương 3.

Từ bối cảnh trên, bài tập lớn đặt ra ba yêu cầu bảo mật cốt lõi cần giải quyết, đúng theo khung yêu cầu của học phần *Nhập môn An toàn Bảo mật Thông tin*:

- (1) **Mã hoá dữ liệu** để đảm bảo tính bảo mật trong quá trình truyền tải dữ liệu cảm biến (siêu âm, NFC) từ ESP32 lên Cloud.
- (2) **Xác thực danh tính thiết bị** để đảm bảo chỉ thiết bị hợp lệ, đã được đăng ký trước, mới có quyền gửi và nhận dữ liệu.
- (3) **Kiểm tra tính toàn vẹn** của dữ liệu để tránh tình trạng dữ liệu bị thay đổi hoặc giả mạo trên đường truyền (tấn công can thiệp giữa đường truyền — MITM).

Bảng 1.1 tổng hợp giải pháp kỹ thuật tương ứng với từng yêu cầu.

Bảng 1.1. Đối chiếu yêu cầu bảo mật và giải pháp áp dụng

Yêu cầu	Giải pháp áp dụng
Bảo mật dữ liệu	Mã hoá đối xứng AES-256-GCM (chế độ mã hoá xác thực — AEAD) [2]
Xác thực danh tính & trao khoá	RSA-1024 — chữ ký số PKCS#1 v1.5 và trao khoá phiên bằng OAEP [3]
Toàn vẹn dữ liệu	Băm SHA-512 [4] kết hợp GCM Tag — hai lớp phòng thủ độc lập
Kênh truyền	Socket TCP qua tên miền thật (mô phỏng triển khai Edge-to-Cloud)

Ngoài ba yêu cầu cốt lõi, đề tài còn yêu cầu mở rộng theo hướng *Authenticated IoT Cloud Ingestion*: mỗi thiết bị phải có định danh (`device_id`) và khoá riêng, dữ liệu phải có timestamp để chống tấn công phát lại (replay attack), và hệ thống phải lưu vết (log) giao dịch theo từng thiết bị.

1.2 Kiến trúc hệ thống

1.2.1 Kiến trúc phần cứng và hạ tầng mạng

Hệ thống được triển khai theo mô hình **Gateway-Node** (Edge-to-Cloud), truyền tải dữ liệu thu thập từ thực địa lên máy chủ trung tâm qua Internet công cộng, thể hiện ở Sơ đồ 1.1.

```

[ ESP32: IoT Edge Node ]
    | Doc cam bien
    v
[ Sieu am HC-SR04 / NFC PN532 ]
    | Ky RSA-1024 (SHA-512) + AES-256-GCM
    | + SessionKey boc bang RSA-OAEP
    v
[ Domain that: iot.yourdomain.com ]
    | Dinh tuyen Internet / Port Forwarding
    v
[ Raspberry Pi 5: Cloud Server ]
    | Xac thuc, giai ma,kiem tra toan ven
    v
[ sensor_data_<device_id>.txt + cloud_transaction.log ]
    |
    v
[ Dashboard web giam sat truc quan ]

```

Sơ đồ 1.1. Kiến trúc Gateway-Node của hệ thống

Nguồn: Nhóm thực hiện tự xây dựng

Vai trò cụ thể của từng thành phần:

- ESP32 (IoT Edge Node):

- *Cảm biến siêu âm (HC-SR04)*: [9] đặt tại cửa, đo khoảng cách liên tục để phát hiện trạng thái có người qua lại/đứng đợi.
- *Mô-đun NFC (PN532)*: [8] đọc UID thẻ (chế độ I2C, chỉ dùng 4 dây VCC/GND/SDA/S — không nối IRQ/RESET vì thư viện điều khiển không bắt buộc) khi cảm biến siêu âm phát hiện có người; UID đọc được gửi kèm gói tin lên Cloud để **Server** — chứ không phải ESP32 — đối chiếu với danh sách thẻ được cấp phép và ra quyết định cho vào/từ chối (mục 2.2.2).
- *Nhiệm vụ*: thu thập dữ liệu, thực hiện các bước bảo mật (ký số, sinh và bọc khoá phiên, mã hoá AES-GCM, băm SHA-512), đóng gói và gửi chuỗi JSON qua Socket TCP. Các thao tác mật mã trên ESP32 sử dụng thư viện mbedTLS tích hợp sẵn trong ESP-IDF/Arduino core [7].

- Raspberry Pi 5 (Cloud Server giả lập):

- Đóng vai trò cụm lưu trữ đám mây giả lập, mở cổng đón kết nối Socket TCP từ Internet.
- *Nhiệm vụ*: lắng nghe kết nối, bóc tách chuỗi JSON, xác thực chữ ký RSA, kiểm tra chống phát lại theo timestamp, giải mã AES-GCM, lưu trữ dữ liệu và ghi log giao dịch theo thời gian thực.
- **Tên miền (Domain)**: cấu hình bản ghi A Record trỏ về IP Public của mạng chứa Raspberry Pi 5; ESP32 kết nối trực tiếp tới tên miền thay vì IP cục bộ, mô phỏng đúng mô hình triển khai Cloud thực tế.
- **Dashboard giám sát**: ứng dụng web (Flask), được `server.py` tự khởi động kèm theo trong một luồng nền — chỉ cần chạy một tiến trình duy nhất là có cả Cloud Server lẫn giao diện giám sát. Dashboard gồm 2 tab: *Giám sát* (trạng thái thiết bị theo thời gian thực kèm biểu đồ xu hướng khoảng cách, kết quả Ra/Vào, bảng nhật ký giao dịch có bộ lọc/tìm kiếm) và *Kịch bản tấn công* (hướng dẫn tái hiện Luồng 2/3, thống kê NACK theo lý do). Một nút trạng thái cho phép **bật/tắt mã hoá ngay khi hệ thống đang chạy**, không cần khởi động lại tiến trình, phục vụ trực tiếp việc trình diễn Luồng 2.

1.2.2 Tham số vận hành và mô hình xử lý đồng thời

Ngoài kiến trúc tổng thể, các tham số vận hành cụ thể (trích từ `iot/include/config.h` và `cloud_server/`) quyết định trực tiếp hành vi thực tế của hệ thống và cần được nêu rõ để đảm bảo khả năng tái lập kết quả thử nghiệm ở Chương 3.

- **Phía ESP32**: chu kỳ đọc cảm biến `SENSOR_POLL_INTERVAL_MS = 500` (đọc siêu âm mỗi 0,5 giây); ngưỡng phát hiện có người `ULTRASONIC_PRESENCE_THRESHOLD_CM = 50` (khoảng cách đo được nhỏ hơn 50 cm mới coi là có người); `MIN_UPLOAD_INTERVAL_MS = 3000` — khoảng cách tối thiểu giữa hai lần gửi liên tiếp, tránh việc một người đứng yên trước cửa khiến thiết bị gửi gói tin liên tục không cần thiết; thời gian chờ tối đa để dò thẻ NFC sau khi phát hiện có người là `PN532_READ_TIMEOUT_MS = 1500`.
- **Phía Cloud Server**: mỗi kết nối Socket có timeout 15 giây (`Handler.timeout`); độ lệch timestamp tối đa cho phép giữa thiết bị và Server là `MAX_TIMESTAMP_SKEW_SECONDS = 30` (mục 2.2.2, Chương 2).
- **Mô hình xử lý đồng thời**: Cloud Server sử dụng `ThreadingTCPServer` (kết hợp `socketserver.ThreadingMixIn`), mỗi kết nối từ một thiết bị được xử lý trên một luồng (thread) riêng, cho phép nhiều ESP32/thiết bị gửi dữ liệu đồng thời mà không

chặn lẫn nhau. Các trạng thái dùng chung giữa các luồng (danh sách khoá công khai thiết bị, mốc timestamp gần nhất đã chấp nhận của từng thiết bị, danh sách thẻ NFC được cấp phép) đều được bảo vệ bằng threading. Lock riêng trong từng lớp quản lý tương ứng, tránh tình trạng tranh chấp dữ liệu (race condition) khi nhiều thiết bị gửi gói tin gần như đồng thời.

1.2.3 Thiết kế cấu trúc gói tin JSON

Toàn bộ thông tin truyền tải qua Socket TCP được định dạng dưới dạng chuỗi JSON, mỗi gói tin kết thúc bằng ký tự xuống dòng (\n) để phân tách các bản tin trên cùng một kết nối. Giao thức gồm ba loại bản tin.

(a) Bắt tay (Handshake):

```
// ESP32 -> Cloud
{ "type": "HELLO", "device_id": "esp32-gate-001" }
// Cloud -> ESP32
{ "type": "READY" }
// hoặc, nếu device_id chưa đăng ký:
{ "type": "REJECT", "reason": "unknown_device" }
```

(b) Gói tin dữ liệu (Upload):

```
{
  "type": "UPLOAD",
  "device_id": "esp32-gate-001",
  "timestamp": 1735900000,
  "metadata": { "sensor_type": "ultrasonic+nfc" },
  "sig": "<Chu ký RSA-1024 PKCS#1v1.5 + SHA-512>",
  "enc_session_key": "<SessionKey bọc bằng RSA-OAEP-SHA256>",
  "nonce": "<Base64, 12 bytes, AES-GCM sinh ngẫu nhiên>",
  "cipher": "<Dữ liệu cảm biến đã mã hóa AES-256-GCM, Base64>",
  "tag": "<GCM Tag, 16 bytes, Base64>",
  "hash": "<SHA-512(nonce||cipher||tag), hex>"
}
```

Cloud phản hồi {"type": "ACK", "status": "ok"} hoặc {"type": "NACK", "reason": "..."} với reason thuộc tập {auth, integrity, replay_detected, timestamp_out_of_range,

unknown_device} — mỗi giá trị tương ứng với một lớp kiểm tra bảo mật được trình bày chi tiết ở Chương 2.

(c) Gói tin tải xuống (Download): chiều ngược lại của (b), dùng khi một bên đã đăng ký muốn lấy lại bản ghi mới nhất của một thiết bị (phân tích chi tiết cơ chế ở mục 2.3, Chương 2).

```
// Ben yeu cau -> Cloud
{
  "type": "DOWNLOAD",
  "device_id": "esp32-gate-001",
  "timestamp": 1735900050,
  "sig": "<Chu ky tren canonical_metadata(requester_id, timestamp, \"download\"...)\"
}
// Cloud -> Ben yeu cau (neu hop le)
{
  "type": "DOWNLOAD_DATA",
  "device_id": "cloud-server",
  "timestamp": 1735900050,
  "metadata": { "sensor_type": "download_response" },
  "sig": "<Chu ky cua chinh Cloud Server>",
  "enc_session_key": "<SessionKey moi, boc bang khoa cong khai cua ben yeu cau...>",
  "nonce": "...", "cipher": "...", "tag": "...", "hash": "..."
}
```

So với thiết kế gói tin tối thiểu ban đầu (chỉ gồm nonce, cipher, tag, hash, sig), gói tin thực tế được bổ sung device_id, timestamp và metadata — đây là phần mở rộng bắt buộc theo yêu cầu nâng cấp *Authenticated IoT Cloud Ingestion*: mỗi thiết bị có định danh riêng, và dữ liệu có mốc thời gian để làm cơ sở chống tấn công phát lại.

Chương 2

THIẾT KẾ THUẬT TOÁN VÀ PHÂN TÍCH MÃ NGUỒN

2.1 Mô tả thuật toán — ba bước bảo mật

Giao thức bảo mật được thiết kế theo đúng ba bước bắt buộc của học phần: bắt tay, xác thực và trao đổi khoá, truyền dữ liệu và kiểm tra toàn vẹn.

2.1.1 Bước 1 — Bắt tay (*Handshake*)

ESP32 mở kết nối Socket, gửi bản tin `{"type": "HELLO", "device_id": ...}` tới tên miền của Cloud Server. Cloud tra cứu `device_id` trong danh sách thiết bị đã đăng ký (khóa công khai RSA tương ứng) — nếu hợp lệ, Raspberry Pi 5 phản hồi `{"type": "READY"}`; nếu không, phản hồi `{"type": "REJECT", "reason": "unknown_device"}` và đóng kết nối ngay. Đây là lớp phòng thủ đầu tiên, chặn thiết bị lạ trước cả khi vào bước ký số và mã hoá, giúp giảm tải xử lý mật mã không cần thiết cho các kết nối không hợp lệ.

2.1.2 Bước 2 — Xác thực (*Ký số và trao đổi khoá*)

ESP32 ký số vào metadata (`device_id`, `timestamp`, loại cảm biến) bằng khoá riêng RSA-1024 của thiết bị, sử dụng lược đồ chữ ký PKCS#1 v1.5 với hàm băm SHA-512 [3][4]. Đồng thời, ESP32 sinh một `SessionKey` ngẫu nhiên (khóa đối xứng AES-256 dùng một lần) rồi mã hoá khoá này bằng khoá công khai RSA-1024 của Server theo lược đồ OAEP.

Giới hạn kỹ thuật quan trọng: đề bài gốc yêu cầu “RSA-1024-bit (OAEP + SHA-512)”. Tuy nhiên, tổ hợp RSA-1024 với OAEP dùng SHA-512 làm hàm băm nội bộ là *không khả thi về mặt toán học*: độ dài đệm tối thiểu của OAEP là $2h + 2$ byte, với h là kích thước hash; SHA-512 có $h = 64$ byte nên đệm tối thiểu là $2 \times 64 + 2 = 130$ byte — vượt quá kích thước modulus RSA-1024 ($k = 128$ byte). Do đó, mọi thư viện mật mã tuân thủ chuẩn [3] (PyCryptodome, mbedTLS, OpenSSL) đều từ chối thao tác này. Nhóm thực hiện xử lý giới hạn trên như sau:

- Hàm băm nội bộ của OAEP (dùng để bọc `SessionKey`) chuyển sang **SHA-256** ($h = 32$ byte, đệm 66 byte, còn dư 62 byte — đủ chứa khoá AES-256 dài 32 byte).
- SHA-512 vẫn được giữ nguyên cho chữ ký số (không bị giới hạn kích thước như OAEP) và cho băm toàn vẹn gói tin ở Bước 3, đúng theo yêu cầu đề bài.

Đây cũng là minh chứng cụ thể cho khuyến nghị của NIST [5] rằng khoá RSA 1024-bit không còn đủ an toàn cho triển khai thực tế dài hạn — không chỉ về độ khó phá vỡ bằng tính toán, mà còn giới hạn cả các lược đệm có thể sử dụng cùng khoá. Nội dung này được phân tích sâu hơn ở mục 3.3.

Về mặt triển khai, hai chế độ đệm nói trên được cấu hình **tách biệt trên cùng một cặp khoá RSA-1024** của mỗi bên: hàm `cryptoInit()` (`iot/src/crypto_utils.cpp`) chỉ gọi `mbedtls_rsa_set_padding(..., MBEDTLS_RSA_PKCS_V21, MBEDTLS_MD_SHA256)` trên ngữ cảnh khoá công khai của Server (dùng để bọc `SessionKey` bằng OAEP), còn ngữ cảnh khoá riêng của thiết bị (dùng để ký) giữ nguyên chế độ đệm mặc định `PKCS#1 v1.5` của `mbedtls_pk_sign()`. Phía Cloud Server (Python), sự tách biệt này thể hiện qua việc `crypto_utils.py` định nghĩa hai cặp hàm độc lập: `sign_data()/verify_signature()` dùng `pkcs1_15` (PyCryptodome), và `rsa_oaep_encrypt()/rsa_oaep_decrypt()` dùng `PKCS1_OAEP` với `hashAlgo=SHA256` tương minh — đảm bảo cả hai nền tảng (mbedtls trên ESP32 và PyCryptodome trên Server) diễn giải cùng một khoá theo đúng hai lược đệm tương ứng, không bị lẫn lộn.

2.1.3 Bước 3 — Truyền dữ liệu và kiểm tra toàn vẹn

- (a) Tạo nonce ngẫu nhiên 12 byte.
- (b) Mã hoá dữ liệu cảm biến (JSON chứa khoảng cách siêu âm, trạng thái có người, UID thẻ NFC) bằng AES-256-GCM [2], thu được cipher và tag xác thực 16 byte.
- (c) Tính hash toàn vẹn: `SHA-512(nonce || cipher || tag)`.
- (d) Gửi gói tin JSON hoàn chỉnh (mục 1.2.2, Chương 1) lên Cloud qua tên miền.

Việc kết hợp GCM Tag (toàn vẹn ở tầng mã hoá, được AES-GCM tự sinh) và SHA-512 (toàn vẹn ở tầng gói tin, tính độc lập trên toàn bộ nonce/cipher/tag) tạo ra **hai lớp phòng thủ độc lập** chống lại tấn công sửa đổi dữ liệu trên đường truyền: nếu kẻ tấn công sửa cipher nhưng giữ nguyên hash và sig gốc (vì không có khoá riêng để ký lại), cả hai lớp kiểm tra phía Server đều phát hiện sai lệch. Cơ chế này được kiểm chứng thực nghiệm ở Luồng 3, Chương 3.

2.2 Phân tích mã nguồn

Mã nguồn được tổ chức thành các mô-đun độc lập theo chức năng, triển khai trên hai nền tảng: firmware ESP32 (C++, PlatformIO/Arduino framework) và máy chủ Cloud (Python).

2.2.1 Firmware ESP32 (thư mục *iot/*)

- **include/config.h** — tập trung toàn bộ tham số cấu hình: định danh thiết bị, thông tin Wi-Fi, địa chỉ/cổng Cloud Server, chân cảm HC-SR04/PN532, ngưỡng phát hiện có người, kích thước khoá/đệm mật mã. Tách cấu hình khỏi logic giúp dễ dàng thay đổi tham số khi triển khai trên phần cứng khác nhau.
- **src/sensors.cpp (khai báo tại include/sensors.h)** — module đọc cảm biến:
 - **sensorsInit()**: khởi tạo chân HC-SR04 và bus I2C cho PN532, kiểm tra phản hồi firmware của PN532.
 - **readUltrasonicDistanceCm()**: phát xung TRIG 10μs rồi đo độ rộng xung phản hồi trên chân ECHO bằng **pulseIn()** (timeout 30ms/lần đo); khoảng cách được tính theo công thức vật lý $d = \frac{t \times 0,0343 \text{ cm}/\mu\text{s}}{2}$ (chia 2 vì xung phải đi và về), lặp lại **ULTRASONIC_SAMPLE_COUNT = 3** lần rồi lấy trung bình cộng các mẫu đo được để lọc nhiễu; nếu cả 3 lần đều timeout (không có vật cản trong tầm đo), hàm trả về -1 để **takeSensorReading()** phân biệt với khoảng cách hợp lệ.
 - **readNfcUid()**: dò thẻ NFC/RFID ISO14443A qua PN532 (chế độ I2C, chỉ 4 dây VCC/GND/SDA/SCL) trong cửa sổ tối đa **PN532_READ_TIMEOUT_MS = 1500ms**, trả về UID dạng chuỗi hex viết hoa (2 ký tự/byte).
 - **takeSensorReading()**: hàm tổng hợp — so sánh khoảng cách đo được với **ULTRASONIC_PRESENCE_THRESHOLD_CM = 50**; chỉ khi khoảng cách nhỏ hơn ngưỡng này (coi là "có người") mới gọi tiếp **readNfcUid()**, giúp tiết kiệm năng lượng và thời gian xử lý — không dò thẻ liên tục khi không có ai trước cửa.
- **src/crypto_utils.cpp (khai báo tại include/crypto_utils.h)** — module mật mã, xây dựng trên thư viện mbedTLS [7] đi kèm ESP32 Arduino core:
 - **cryptoInit()**: nạp khoá riêng của thiết bị và khoá công khai của Server từ PEM, cấu hình chế độ đệm OAEP/SHA-256 riêng cho ngữ cảnh bọc khoá phiên (**mbedtls_rsa_set_padding**).
 - **signWithDeviceKey()**: băm SHA-512 rồi ký PKCS#1 v1.5 bằng khoá riêng thiết bị (**mbedtls_pk_sign**).
 - **wrapSessionKeyForServer()**: mã hoá SessionKey bằng OAEP với khoá công khai Server (**mbedtls_pk_encrypt**).
 - **aesGcmEncrypt()**: mã hoá AES-256-GCM, sinh đồng thời cipher và tag (**mbedtls_gcm_crypt_and_tag**).

- `sha512Hash()`, `secureRandomBytes()`, `base64Encode()/base64Decode()`: các hàm hỗ trợ băm, sinh số ngẫu nhiên bằng bộ sinh số cứng của ESP32, và mã hoá Base64 cho các trường nhị phân trong gói JSON.
- `src/main.cpp` — điều phối luồng chính: kết nối Wi-Fi, đồng bộ giờ thực qua NTP (phục vụ chống phát lại), khởi tạo mật mã và cảm biến, và vòng lặp `loop()` thực hiện: đọc cảm biến → nếu phát hiện có người và đã đủ khoảng cách thời gian tối thiểu kể từ lần gửi trước → gọi `uploadReading()` để thực hiện tuần tự ba bước bảo mật ở mục 2.1 và gửi gói tin qua `WiFiClient`.

2.2.2 Cloud Server (thư mục `cloud_server/`)

- `crypto_utils.py` — đối xứng với mô-đun mật mã phía ESP32, dùng thư viện PyCryptodome: `sign_data()/verify_signature()` (RSA-PKCS1v15+SHA-512), `rsa_oaep_encrypt()/rsa_oaep_decrypt()` (RSA-OAEP+SHA-256), `aes_gcm_encrypt/integrity_hash()` (SHA-512 trên `nonce|cipher|tag`), và `canonical_metadata()` — hàm dựng chuỗi canonical dùng để ký/xác thực, phải khớp tuyệt đối giữa ESP32 và Server.
- `device_registry.py` — lớp `DeviceRegistry` quản lý danh sách khoá công khai thiết bị đã đăng ký (nạp từ thư mục `keys/devices/`) và trạng thái chống phát lại: `check_timestamp()` so sánh timestamp gói tin với mốc thời gian gần nhất đã chấp nhận của từng thiết bị, đồng thời giới hạn độ lệch tối đa so với giờ hệ thống Server (mặc định 30 giây) để phát hiện timestamp bất thường.
- `storage.py` — ghi dữ liệu cảm biến đã giải mã vào `sensor_data_<device_id>.txt` (một file riêng cho mỗi thiết bị) và log giao dịch (ACK/NACK kèm lý do, thời gian) vào `cloud_transaction.log` dạng JSON theo dòng, phục vụ đối chiếu ở Chương 3.
- `access_control.py` — lớp `AccessControl` quản lý danh sách UID thẻ NFC được cấp phép (nạp từ `keys/authorized_cards.json`). Hàm `evaluate(nfc_detected, nfc_uid)` trả về cặp (`access_granted`, `access_reason` ∈ {`no_card`, `card_authorized`, `card_unauthorized`}) — đây là lớp **phân quyền (authorization)**, tách biệt với các lớp xác thực/mã hoá/toàn vẹn: một gói tin có thể được Server chấp nhận (ACK, chữ ký và toàn vẹn hợp lệ) nhưng vẫn bị **từ chối ra/vào** nếu UID quét được không nằm trong danh sách cấp phép. Quyết định này được áp dụng ở *cả hai* chế độ SECURE và baseline, nhằm minh chứng rõ ràng ở Luồng 2 rằng thiếu xác thực khiến quyết định "cho vào" cũng bị giả mạo theo (Chương 3).

- `server.py` — máy chủ Socket TCP đa luồng (`ThreadingTCPServer`), triển khai đúng trình tự kiểm tra đã mô tả ở mục 2.1: xác thực `device_id` ở bước bắt tay → kiểm tra `timestamp` (chống phát lại/bất thường) → xác thực chữ ký → kiểm tra hash toàn vẹn → giải mã `SessionKey` và AES-GCM → xác minh quyền ra/vào qua `AccessControl` → lưu trữ và ghi log. Bất kỳ bước xác thực/toàn vẹn nào thất bại đều trả về NACK kèm lý do cụ thể, không tiếp tục xử lý các bước sau; kết quả `access_granted` được gửi kèm trong bản tin ACK. Server còn hỗ trợ chế độ `--baseline`, vô hiệu hoá toàn bộ xác thực/mã hoá để phục vụ minh hoạ lỗi hổng ở Luồng 2 (Chương 3); cờ chế độ này (`ServerContext.baseline`) có thể được **bật/tắt ngay khi server đang chạy** thông qua endpoint `POST /api/mode` của dashboard, không cần khởi động lại tiến trình — khoá Server và danh sách thiết bị luôn được nạp sẵn ở cả hai chế độ để đảm bảo việc chuyển đổi qua lại an toàn. Server cũng hỗ trợ luồng tải xuống (*download*, mục 2.3), cho phép thiết bị/người dùng lấy lại dữ liệu đã lưu, được Server ký số và mã hoá lại bằng khoá công khai của bên yêu cầu trước khi gửi trả.

Nhận xét về thứ tự kiểm tra trong `_handle_upload()`: thứ tự (1) kiểm tra `timestamp` trước rồi mới (2) xác thực chữ ký RSA không phải ngẫu nhiên. Kiểm tra `timestamp` chỉ là một phép so sánh số nguyên và tra cứu dict trong bộ nhớ (chi phí $O(1)$), trong khi xác thực chữ ký RSA-1024 là phép toán mũ modulo tốn nhiều chu kỳ CPU hơn đáng kể. Đặt bước rẻ trước bước đắt giúp Server từ chối sớm các gói tin phát lại (vốn chiếm phần lớn lưu lượng tấn công tự động) mà không phải tốn tài nguyên xác minh chữ ký cho những gói tin chắc chắn sẽ bị loại — một hình thức phòng thủ chống *tấn công từ chối dịch vụ* (DoS) ở mức nhỏ, dù không phải mục tiêu chính của đề tài.

2.2.3 Luồng tải xuống (*Download*) — chiều ngược của giao thức

Bên cạnh luồng tải lên (*upload*) đã trình bày ở mục 2.1, hàm `_handle_download()` trong `server.py` triển khai chiều ngược lại: cho phép một bên đã đăng ký (thiết bị hoặc công cụ quản trị) lấy về bản ghi dữ liệu mới nhất của một thiết bị bất kỳ đã lưu trên Cloud, với cùng mức bảo vệ như luồng tải lên.

- (a) **Yêu cầu:** bên yêu cầu (`requester_id`) gửi bản tin `{"type": "DOWNLOAD", "device_id": "<id>"}` trong đó `sig` là chữ ký RSA-PKCS1v15/SHA-512 trên cùng dạng canonical đã dùng ở luồng upload, nhưng với `sensor_type` cố định là chuỗi `"download"` — tức là `canonical_metadata(requester_id, timestamp, "download")`. Việc tái sử dụng đúng hàm `canonical_metadata()` (thay vì tạo định dạng chữ ký riêng cho luồng

download) giúp giảm bề mặt tấn công: chỉ có một điểm duy nhất trong mã nguồn định nghĩa "cái gì được ký".

- (b) **Xác thực yêu cầu:** Server kiểm tra `timestamp` của chính `requester_id` qua `DeviceRegister` (chống phát lại yêu cầu tải xuống) rồi xác thực `sig` bằng khoá công khai đã đăng ký của `requester_id` — nếu một trong hai bước thất bại, trả NACK với lý do tương ứng (`replay_detected/timestamp_out_of_range/auth`) mà **không tiết lộ** dữ liệu của thiết bị đích có tồn tại hay không.
- (c) **Đóng gói phản hồi:** nếu yêu cầu hợp lệ, Server đọc dòng cuối cùng của `sensor_data_<device_id>` tương ứng, rồi tự đóng vai "bên gửi": sinh một `SessionKey` AES-256 mới bằng `os.urandom()`, mã hoá dữ liệu bằng AES-GCM, tính lại SHA-512(`nonce||cipher||tag`), **tự ký** bản tin bằng chính khoá riêng của Server (với `device_id` khai báo là "cloud-server"), và bọc `SessionKey` mới bằng khoá công khai của bên yêu cầu (không phải khoá Server) — đảm bảo chỉ bên đã gửi yêu cầu mới giải mã được. Gói phản hồi có type là `DOWNLOAD_DATA`, cấu trúc trường hoàn toàn tương tự gói `UPLOAD` ở mục 1.2.2.

Như vậy, luồng download không phải là một kênh "đọc dữ liệu" tách rời, mà tái sử dụng *nguyên vẹn* bộ nguyên thủy mật mã (ký số, OAEP, AES-GCM, băm toàn vẹn) đã dùng cho luồng upload, chỉ đảo ngược vai trò gửi/nhận — Server trở thành "người ký" và "người mã hoá", còn bên yêu cầu là "người xác minh" và "người giải mã". Đây là điểm thiết kế quan trọng cần nêu trong báo cáo vì nó cho thấy giao thức được xây dựng đối xứng hai chiều, không chỉ bảo vệ dữ liệu đi lên Cloud mà cả dữ liệu đi xuống thiết bị/người dùng.

2.2.4 Công cụ hỗ trợ

- `tools/generate_keys.py`, `tools/pem_to_header.py` — sinh cặp khoá RSA-1024 cho Server và từng thiết bị; chuyển đổi khoá PEM thành khai báo chuỗi C++ để nhúng vào firmware.
- `device_simulator/simulate_device.py` — mô phỏng hành vi của một thiết bị ESP32 bằng Python, triển khai cùng giao thức mật mã, cho phép kiểm thử toàn bộ pipeline mà không cần phần cứng thật (bao gồm cả các kịch bản lỗi: sửa dữ liệu, phát lại gói tin, timestamp bất thường).
- `attacker_sim/` — ba kịch bản tấn công đối chứng, đều nhắm vào Server chạy chế độ `--baseline` hoặc chặn giữa đường truyền: `baseline_fake_packet.py` (giả vờ *không có ai* đi qua để che giấu xâm nhập), `baseline_fake_access.py` (giả mạo *có thể NFC hợp lệ* — tự khai một UID nằm trong danh sách cấp phép — để được hệ thống "cho vào" dù không hề đứng trước cảm biến, Luồng 2), và `mitm_tamper.py`

— proxy TCP đứng giữa để chặn và sửa trường cipher của gói tin hợp lệ trước khi chuyển tiếp lên Server thật (Luồng 3, mô phỏng tấn công MITM).

- **dashboard/** — ứng dụng web (Flask), được `server.py` tự khởi động kèm theo (mục 1.2.1). Đọc trực tiếp dữ liệu đã lưu và log giao dịch, hiển thị trạng thái theo từng thiết bị (kèm biểu đồ xu hướng khoảng cách dạng sparkline), thống kê ACK/NACK theo lý do, kết quả Ra/Vào, và cung cấp nút bật/tắt mã hoá trực tiếp trên giao diện để phục vụ trình diễn Luồng 2 mà không cần thao tác dòng lệnh. Dữ liệu tự làm mới mỗi giây, khớp chu kỳ đọc cảm biến của ESP32.

Ghi chú: số liệu đo hiệu năng thực tế (thời gian mã hoá/ký theo kích thước gói tin) và trích đoạn log giao dịch thật được trình bày cùng kết quả thử nghiệm ở Chương 3, sau khi chạy công cụ kiểm thử tự động trên hệ thống triển khai thực tế.

Chương 3

THỬ NGHIỆM, ĐÁNH GIÁ VÀ ĐỀ XUẤT CẢI TIẾN

3.1 Thử nghiệm — ba kịch bản đối chứng

Để chứng minh tính hiệu quả của giải pháp an toàn thông tin và tạo chiều sâu nghiên cứu, hệ thống được thử nghiệm qua ba kịch bản đối chứng trực quan, sử dụng `device_simulator/` (mô phỏng thiết bị) và `attacker_sim/` (mô phỏng tấn công) đã trình bày ở Chương 2.

3.1.1 Luồng 1 — Hệ thống hoạt động hợp pháp (*Happy Path*)

Kịch bản: Người dùng quét thẻ NFC hợp lệ và đi qua cửa (cảm biến siêu âm xác nhận có người).

Tiến trình xử lý: (1) Handshake — ESP32 gửi HELLO kèm `device_id`, Server xác nhận thiết bị đã đăng ký và phản hồi READY; (2) Xác thực & trao khoá — ESP32 ký số metadata và bọc SessionKey bằng khoá công khai Server; (3) Mã hoá & toàn vẹn — mã hoá dữ liệu cảm biến bằng AES-GCM, băm SHA-512, gửi gói tin JSON hoàn chỉnh.

Kết quả mong đợi: Server xác thực chữ ký hợp lệ, kiểm tra toàn vẹn khớp, giải mã thành công dữ liệu siêu âm/NFC, lưu file `sensor_data_<device_id>.txt`, trả ACK và ghi log giao dịch thành công. Ngoài ra, Server đối chiếu UID thẻ NFC nhận được với danh sách thẻ được cấp phép (`access_control.py`, mục 2.2.2): nếu khớp, trả thêm `access_granted=true` (*Cho vào*); nếu quét thẻ không có trong danh sách, hoặc siêu âm phát hiện có người nhưng không quét thẻ, hệ thống vẫn ACK (gửi tin hợp lệ, xác thực được) nhưng `access_granted=false` (*Từ chối*) — thể hiện rõ ràng rằng "xác thực được nguồn gốc dữ liệu" và "được phép ra/vào" là hai quyết định độc lập.

3.1.2 Luồng 2 — Hệ thống không bảo mật (trường hợp đối chứng — *Baseline*)

Mục đích: Chứng minh lỗ hổng nguy hiểm nếu hệ thống IoT kiểm soát ra vào truyền tin thô, không mã hoá — đúng như phân tích ở mục 1.1. Chế độ này được bật/tắt ngay trên dashboard (nút trạng thái mã hoá, mục 2.2.2) mà không cần khởi động lại Server, giúp trình diễn nhanh và trực quan.

Kịch bản (a) — Che giấu xâm nhập: Kẻ xấu lén qua cửa. Để che giấu hành vi, kẻ xấu dùng máy tính gửi trực tiếp một gói tin JSON thô (plaintext), không mã hoá, lên Cloud

Server (chế độ --baseline, vô hiệu hoá toàn bộ xác thực), qua attacker_sim/baseline_fake_pa

```
{ "status": "Khong co ai ra vao", "rfid": "None" }
```

Kết quả mong đợi: Do chế độ baseline không chạy bất kỳ thuật toán bảo mật nào, Cloud Server chấp nhận dữ liệu giả mạo này, cập nhật trạng thái bình thường và không đưa ra cảnh báo. Kẻ xấu đột nhập thành công mà không để lại vết.

Kịch bản (b) — Giả mạo quyền ra/vào: Ngược lại với kịch bản (a), kẻ xấu *không* hệ đứng trước cảm biến (presence=false) nhưng tự khai báo trong gói tin gửi thẳng lên Server rằng đã quét một thẻ NFC hợp lệ (nfc_detected=true, UID trùng với một thẻ có trong authorized_cards.json), qua attacker_sim/baseline_fake_access.py:

```
{ "distance_cm": 999.0, "presence": false,
  "nfc_detected": true, "nfc_uid": "<UID hợp lệ>" }
```

Kết quả mong đợi: Vì chế độ baseline không xác thực chữ ký nên không thể phân biệt UID này là do thiết bị thật đọc được hay do kẻ tấn công tự khai báo — lớp access_control.py (vốn hoạt động độc lập với lớp mã hoá) vẫn trả access_granted=true, hệ thống ”mở cửa” dù không có ai thật sự đứng trước cảm biến.

Nhận xét chung: hai kịch bản trên minh chứng **hai mặt của cùng một lỗ hổng gốc** — thiếu xác thực chữ ký khiến Server không có cơ sở nào để tin hay không tin nội dung gói tin, dù mục tiêu giả mạo là che giấu hành vi (a) hay giả mạo quyền truy cập (b). Đây cũng là ý tưởng triển khai ban đầu của nhóm khi thiết kế kịch bản đối chứng: hình dung một kẻ tấn công có thể ”hack được gửi dữ liệu lên server dù có người lại gần nhưng cảm biến siêu âm không quét ra ai, hacker quét thẻ fake vẫn cho vào” — chính là kịch bản (b) ở trên.

3.1.3 Luồng 3 — Kịch bản tấn công can thiệp nâng cao (bẻ gãy đòn tấn công)

Mục đích: Kiểm thử khả năng phòng thủ khi kẻ tấn công có trình độ can thiệp trực tiếp vào luồng bảo mật (tấn công đứng giữa — MITM).

Kịch bản: ESP32 chính thống phát hiện người qua cửa và gửi gói tin bảo mật hợp lệ. Kẻ xấu dùng proxy đứng giữa kênh truyền (attacker_sim/mitm_tamper.py) bắt gói tin JSON, nhận thấy chữ ký số (sig) hợp lệ nên giữ nguyên, nhưng cố tình sửa 1 byte của trường cipher nhằm giả mạo kết quả, rồi chuyển tiếp gói tin đã sửa lên Cloud Server thật.

Cơ chế phòng thủ của Server:

- (1) Server xác thực chữ ký RSA — khớp, vì kẻ tấn công không đụng tới sig/metadata.

- (2) Server tự tính lại $\text{SHA-512}(\text{nonce} \parallel \text{cipher} \parallel \text{tag})$ — lệch hoàn toàn so với trường hash đi kèm, vì cipher đã bị sửa.
- (3) Nếu vẫn tiếp tục giải mã, mã xác thực AES-GCM Tag cũng lập tức báo lỗi toàn vẹn.

Kết quả mong đợi: Server phát hiện dữ liệu bị can thiệp, huỷ giao dịch, trả NACK với `reason="integrity"`, và ghi nhận lỗi vào log. Kẻ tấn công bị lộ mặt dù giữ được chữ ký hợp lệ — minh chứng cho hiệu quả của cơ chế **hai lớp phòng thủ độc lập** đã thiết kế ở Chương 2.

3.1.4 Bộ kiểm thử tự động (*tests/test_end_to_end.py*)

Ngoài ba kịch bản trực quan ở trên, nhóm xây dựng một bộ kiểm thử tự động khởi động thật một tiến trình Cloud Server (chế độ SECURE, cổng cô lập riêng để không ảnh hưởng dữ liệu demo), rồi lần lượt gọi `device_simulator/simulate_device.py` với các tham số khác nhau và đối chiếu phản hồi. Bảng ?? liệt kê đầy đủ 8 phép kiểm tra (hàm `check()`) mà bộ kiểm thử thực hiện.

Bảng 3.1. Danh sách ca kiểm thử tự động và kết quả mong đợi

Ca	Mô tả / cách tạo	Kết quả mong đợi
1	Thiết bị hợp lệ, gửi upload bình thường	ACK, status=ok
2	Thiết bị giả mạo: sinh một cặp khoá RSA-1024 mới, hoàn toàn không có trong keys/devices/, rồi ký gói tin bằng khoá đó với device_id="esp32-fake-999"	REJECT, reason=unknown_device (bị chặn ngay ở bước bắt tay, mục 2.1.1)
3	--tamper-cipher: giả lập MITM, sửa 1 byte trường cipher trước khi gửi	NACK, reason=integrity
4a	Gửi gói tin hợp lệ đầu tiên, đồng thời lưu lại nguyên văn gói tin ra file (--save-packet)	ACK, status=ok
4b	Gửi lại (replay) y nguyên gói tin đã lưu ở ca 4a bằng --replay-file	NACK, reason=replay_detected
5	--timestamp-offset 999999: đẩy timestamp lệch tới gần 12 ngày so với giờ thực	NACK, reason=timestamp_out_of_range
6	Kiểm tra tồn tại file sensor_data_esp32-gate-001.txt sau ca 1	File log riêng theo thiết bị được tạo đúng
7	Kiểm tra tồn tại và không rỗng của cloud_transaction.log	Log giao dịch được ghi liên tục theo thời gian thực

Điểm đáng chú ý ở ca 2: gói tin của esp32-fake-999 có chữ ký *hợp lệ về mặt toán học* — được ký đúng bằng khoá riêng tương ứng với khoá công khai mà kẻ tấn công tự sinh — nhưng vẫn bị từ chối ngay từ bước bắt tay (HELLO) vì device_id không có trong DeviceRegistry. Ca này minh chứng rằng lớp **xác thực danh tính bằng danh sách đăng ký trước** (mục 2.1.1, Chương 2) chặn được cả những kẻ tấn công am hiểu mật mã và có khả năng tự tạo cặp khoá riêng, chứ không chỉ chặn được lỗi định dạng gói tin.

3.2 Đánh giá hiệu quả

- **Tính đúng đắn:** ở Luồng 1, dữ liệu siêu âm/NFC sau khi giải mã tại Server trùng khớp hoàn toàn với dữ liệu gốc trước khi mã hoá tại ESP32.
- **Tính toàn vẹn:** ở Luồng 3, hệ thống phát hiện chính xác các trường hợp dữ liệu bị chỉnh sửa trên đường truyền (SHA-512 và GCM Tag đều sai lệch), từ chối giải mã và trả NACK.
- **Rủi ro khi thiếu bảo mật:** ở Luồng 2, hệ thống baseline không có cơ chế bảo vệ chấp nhận hoàn toàn dữ liệu giả mạo, ở cả hai chiều — che giấu xâm nhập (kịch bản a) lẫn giả mạo quyền ra/vào (kịch bản b) — cho thấy tính bắt buộc của việc áp dụng mã hoá/xác thực/toàn vẹn trong triển khai thực tế.
- **Phân quyền độc lập với xác thực:** lớp `access_control.py` chứng minh rằng "xác thực được nguồn gốc" (ACK) và "được phép ra/vào" (`access_granted`) là hai lớp bảo vệ tách biệt — một gói tin có chữ ký hợp lệ vẫn có thể bị từ chối ra/vào nếu UID không nằm trong danh sách cấp phép, và ngược lại, nếu thiếu xác thực (Luồng 2), lớp phân quyền một mình không đủ để ngăn giả mạo.
- **Chống phát lại và timestamp bất thường:** công cụ `device_simulator` hỗ trợ trực tiếp các ca kiểm thử gửi lại gói tin cũ (`--replay-file`) và làm lệch timestamp (`--timestamp-offset`); Server phản hồi tương ứng `NACK reason=replay_detected` và `NACK reason=timestamp_out_of_range`.

Hiệu năng: thời gian mã hoá/giải mã AES-GCM và ký/xác thực RSA theo kích thước gói tin khác nhau, đo bằng `tools/generate_test_report.py` (script tự động chạy đủ ba luồng và các ca kiểm thử bắt buộc, ghi kết quả thật vào log giao dịch). Cụ thể, `device_simulator/simulate_device.py` đo thời gian bằng `time.perf_counter()` quanh từng bước bảo mật và in ra một dòng TIMING: `{...}` dạng JSON với 4 trường: `rsa_sign_ms` (ký metadata), `rsa_oaep_wrap_ms` (bọc SessionKey), `aes_gcm_encrypt_ms` (mã hoá dữ liệu cảm biến), và `roundtrip_ms` (tổng thời gian từ lúc kết nối tới lúc nhận được ACK/NACK) — đây chính là 4 cột số liệu cần điền vào Bảng 3.1 bên dưới sau khi chạy kiểm thử thật.

Bảng 3.2. Thời gian xử lý mật mã theo kích thước gói tin (điền số liệu thực đo)

Kích thước payload	AES-GCM (ms)	Ký + bọc khoá RSA (ms)	Round-trip (ms)
Gói nhỏ (~100 byte)	<i>[chờ số liệu]</i>	<i>[chờ số liệu]</i>	<i>[chờ số liệu]</i>
Gói lớn hơn (~4–5 KB)	<i>[chờ số liệu]</i>	<i>[chờ số liệu]</i>	<i>[chờ số liệu]</i>

Ghi chú: bảng trên sẽ được cập nhật bằng số liệu đo thực tế và trích đoạn `cloud_transaction` sau khi hệ thống được triển khai và chạy kiểm thử trên phần cứng thật.

3.3 Phân tích, nhận xét đặc điểm các thuật toán sử dụng

Bảng 3.2 tổng hợp ưu điểm và hạn chế của từng thuật toán được sử dụng để xây dựng hệ thống.

Bảng 3.3. Đặc điểm các thuật toán sử dụng

Thuật toán	Vai trò	Ưu điểm	Hạn chế
AES-GCM [2]	Mã hoá dữ liệu cảm biến	Tốc độ nhanh, tự sinh Tag bảo vệ toàn vẹn (mã hoá xác thực — AEAD)	Cần quản lý nonce chặt chẽ, tuyệt đối không tái sử dụng cùng khoá
RSA-1024 (PKCS#1v1.5 & OAEP) [3]	Ký số & trao đổi khoá phiên	Bảo vệ danh tính thiết bị, chống giả mạo	Độ dài khoá 1024-bit không còn đủ an toàn cho triển khai dài hạn [5]; ràng buộc kích thước khiến OAEP không thể dùng SHA-512 (mục 2.1.2); tốn tài nguyên tính toán trên vi điều khiển
SHA-512 [4]	Băm kiểm tra toàn vẹn, ký số	Khoá chặt toàn vẹn gói tin, chống va chạm mạnh	Chi phí tính toán cao hơn SHA-256 trên thiết bị hạn chế tài nguyên

Điểm đáng chú ý nhất là giới hạn toán học giữa RSA-1024 và OAEP-SHA512 (phân tích chi tiết ở mục 2.1.2): đây không chỉ là vấn đề ”độ an toàn” theo nghĩa thông thường (số phép tính cần để phá vỡ khoá) mà còn là giới hạn cấu trúc — một số tổ hợp thuật toán *không thể triển khai được* nếu không tăng kích thước khoá. Đây là bài học thực nghiệm quan trọng khi thiết kế hệ thống mật mã: các tham số (kích thước khoá, hàm băm, chế độ đệm) phải được lựa chọn đồng bộ, không thể áp dụng máy móc yêu cầu ở mức thuật toán riêng lẻ.

3.4 Đề xuất cải tiến và hướng phát triển

- (1) Chuyển thuật toán bất đối xứng từ RSA sang **đường cong Elliptic (ECC)**, ví dụ ECDSA cho chữ ký và ECDH cho trao khoá, để giảm tải tính toán và cho phép dùng SHA-512 trọn vẹn mà không gặp giới hạn kích thước khoá như RSA-1024.
- (2) Nâng cấp độ dài khoá RSA lên tối thiểu 2048-bit [5] nếu vẫn giữ RSA, để vừa đáp ứng chuẩn an toàn thực tế, vừa đủ chỗ dùng OAEP-SHA512 đúng nguyên văn yêu cầu đề bài.
- (3) Tích hợp camera AI chụp ảnh đối tượng khi phát hiện tấn công (Luồng 3), tự động gửi cảnh báo kèm ảnh qua kênh thông báo tức thời (ví dụ Telegram Bot) khi Server trả NACK.
- (4) Mở rộng dashboard giám sát hiện tại thành hệ thống cảnh báo thời gian thực (real-time alerting) với ngưỡng cấu hình được, thay vì chỉ hiển thị thụ động.
- (5) Bổ sung xác thực hai chiều (mutual TLS) ở tầng vận chuyển bên cạnh giao thức ứng dụng hiện có, để bảo vệ cả siêu dữ liệu (địa chỉ IP, thời điểm kết nối) khỏi bị quan sát trên đường truyền.

KẾT LUẬN

Bài tập lớn đã hoàn thành việc thiết kế và triển khai một hệ thống Gateway-Node giả lập giám sát ra vào bảo mật qua Cloud, đáp ứng đầy đủ ba yêu cầu bảo mật cốt lõi của học phần *Nhập môn An toàn Bảo mật Thông tin*: mã hoá dữ liệu (AES-256-GCM), xác thực danh tính thiết bị (chữ ký số RSA-1024/SHA-512), và kiểm tra tính toàn vẹn dữ liệu (hai lớp phòng thủ độc lập kết hợp GCM Tag và SHA-512). Hệ thống cũng đáp ứng các yêu cầu mở rộng của *Authenticated IoT Cloud Ingestion*: định danh và khoá riêng theo từng thiết bị, chống tấn công phát lại bằng timestamp, xác minh quyền ra/vào theo danh sách thẻ NFC được cấp phép (độc lập với lớp xác thực/mã hoá), và ghi log giao dịch theo từng thiết bị qua cả file log lẫn dashboard giám sát trực quan — dashboard này được tích hợp chạy kèm Cloud Server trong cùng một tiến trình, có khả năng bật/tắt mã hoá ngay khi đang vận hành để phục vụ trình diễn kịch bản đối chứng.

Ba kịch bản đối chứng thực nghiệm (hoạt động hợp pháp, baseline không bảo mật, tấn công can thiệp MITM) đã chứng minh trực quan tính cần thiết của việc áp dụng đầy đủ ba lớp bảo mật: nếu thiếu vắng, kẻ tấn công có thể đột nhập mà không để lại dấu vết; nếu áp dụng đầy đủ, hệ thống phát hiện và từ chối chính xác mọi hành vi giả mạo hoặc can thiệp dữ liệu, kể cả khi kẻ tấn công giữ được chữ ký số hợp lệ.

Quá trình thực hiện cũng cho thấy một giới hạn kỹ thuật quan trọng không được nêu rõ trong đề bài gốc: tổ hợp RSA-1024 với OAEP-SHA512 là bất khả thi về mặt toán học, đòi hỏi phải điều chỉnh hàm băm nội bộ của OAEP sang SHA-256 trong khi vẫn giữ nguyên SHA-512 cho các mục đích khác. Đây là một bài học thực nghiệm có giá trị về việc lựa chọn tham số mật mã đồng bộ, và là cơ sở cho đề xuất nâng cấp lên khoá RSA dài hơn hoặc chuyển sang đường cong Elliptic trong hướng phát triển tiếp theo.

Nhìn chung, hệ thống đã đạt được mục tiêu vận dụng các thuật toán mã hoá và xác thực đã học vào một bài toán thực tế có ý nghĩa (kiểm soát ra vào IoT), đồng thời rèn luyện kỹ năng thiết kế giao thức bảo mật hoàn chỉnh, triển khai đa nền tảng (firmware nhúng và máy chủ Cloud) và tư duy kiểm thử đối chứng trong an toàn thông tin.

PHỤ LỤC

Phụ lục 1. Đối chiếu với yêu cầu báo cáo của học phần

Bảng dưới đối chiếu từng yêu cầu bắt buộc của báo cáo (theo hướng dẫn học phần [1]) với phần tương ứng trong tài liệu này.

Bảng .4. Đối chiếu yêu cầu báo cáo với cấu trúc tài liệu

Yêu cầu báo cáo	Phần tương ứng
1. Đặt vấn đề và phân tích yêu cầu	Mục 1.1
2. Mô tả thuật toán (Handshake, Xác thực, Truyền dữ liệu & toàn vẹn)	Mục 2.1
3. Phân tích mã nguồn	Mục 2.2
4. Thử nghiệm, ghi log thời gian, ghi nhận lỗi	Mục 3.1
5. Đánh giá hiệu quả	Mục 3.2
6. Phân tích, nhận xét đặc điểm thuật toán	Mục 3.3
7. Đề xuất cải tiến và hướng phát triển	Mục 3.4

Phụ lục 2. Mã nguồn và dữ liệu kèm theo

Toàn bộ mã nguồn, dữ liệu mẫu và log giao dịch của hệ thống được tổ chức trong repository `secure-iot-gateway/`, bao gồm:

- `iot/` — Firmware ESP32 (PlatformIO).
- `cloud_server/` — Máy chủ Cloud (Python).
- `dashboard/` — Giao diện giám sát trực quan (Flask).
- `device_simulator/`, `attacker_sim/` — Công cụ mô phỏng thiết bị và kịch bản tấn công đối chứng.
- `tools/` — Công cụ sinh khoá và sinh báo cáo kiểm thử tự động.
- `tests/test_report.md` — Bảng kết quả kiểm thử chi tiết (điền số liệu thực khi chạy `tools/generate_test_report.py`).

- `data/samples/`, `data/logs/` — Dữ liệu cảm biến mẫu và log giao dịch sinh ra khi vận hành hệ thống.

Phụ lục 3. Hình ảnh minh hoạ

Chèn ảnh chụp màn hình Serial Monitor ESP32, terminal Cloud Server, và giao diện Dashboard trong quá trình chạy thử nghiệm thực tế.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] Khoa Công nghệ thông tin, Trường Đại học Đại Nam. *Hướng dẫn thực hiện bài tập lớn học phần Nhập môn An toàn Bảo mật Thông tin — Đề tài 10: Giả lập upload/download dữ liệu IoT lên Cloud với log thời gian*. Học kỳ III, Năm học 2025–2026.
- [2] Dworkin, M. *NIST Special Publication 800-38D: Recommendation for Block Cipher Modes of Operation — Galois/Counter Mode (GCM) and GMAC*. National Institute of Standards and Technology, 2007.
- [3] Moriarty, K., Kaliski, B., Jonsson, J., & Rusch, A. *RFC 8017: PKCS #1 — RSA Cryptography Specifications Version 2.2*. Internet Engineering Task Force (IETF), 2016.
- [4] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. U.S. Department of Commerce, 2015.
- [5] Barker, E. *NIST Special Publication 800-57 Part 1 Revision 5: Recommendation for Key Management — Part 1: General*. National Institute of Standards and Technology, 2020.
- [6] OWASP Foundation. *OWASP Internet of Things Top 10*. 2018.
- [7] Espressif Systems. *ESP32 Technical Reference Manual và mbedTLS API Documentation (ESP-IDF Programming Guide)*. 2024.
- [8] NXP Semiconductors. *PN532/C1 — Near Field Communication (NFC) Controller, User Manual UM0701-02*. 2018.
- [9] Cytron Technologies / ITead Studio. *HC-SR04 Ultrasonic Ranging Module — User Guide*.